

ChatDBG-MTP: Extending AI-Assisted Debugging with Post-Mortem Analysis, Repository-Aware Context, and Automated Repair

Siddhi Tiwari

Dept. of Computer Science & Engineering

[Your Institution], [City, Country]

siddhitiwari0109@gmail.com

Abstract—Debugging is one of the most time-consuming activities in software development. Existing AI-assisted debugging tools, including ChatDBG, require an active debugger session, limiting their applicability to crashes reproducible in a live environment. This paper presents ChatDBG-MTP, which introduces five contributions: (1) *post-mortem analysis* from crash log files, (2) a *zero-friction exception hook* via `sys.excepthook`, (3) *repository-aware context* through import-graph traversal, (4) *git-aware context* using `log/blame/diff`, and (5) *automated fix application and test generation*. Evaluating on 8 benchmarks across four variants, repository context alone improves root-cause keyword coverage from 57 % to 77 % (+20 pp) and fix correctness from 50 % to 75 % (+25 pp). The full *+Repo+Git* configuration achieves 89 % keyword coverage and 88 % fix correctness at only a 38 % cost increase over the baseline.

Index Terms—debugging, large language models, post-mortem analysis, automated program repair, fault localization, git-aware debugging, test generation

I. INTRODUCTION

MODERN software crashes in many contexts where a live debugger is unavailable: production environments, CI pipelines, and remote machines. Even when a debugger is available, AI assistants such as ChatDBG [1] only analyze the immediate stack trace, missing the broader codebase context that could accelerate diagnosis.

We identify three key gaps: (i) *Availability*—tools require a live process; (ii) *Context*—analysis is limited to the call stack, ignoring imported modules and recent code changes; (iii) *Completeness*—tools explain bugs but do not apply fixes or write regression tests.

ChatDBG-MTP addresses all three through: a post-mortem mode (`chatdbg --analyze crash.log`), a zero-friction exception hook (`chatdbg.install()`), repository context (`--repo`), git context (`automatic`), and an automated repair pipeline (`apply_fix`, `generate_test`).

II. BACKGROUND AND RELATED WORK

A. ChatDBG

ChatDBG [1] integrates an LLM into Python `pdb/ipdb` and native GDB/LLDB. When the user types `why`, it sends

the stack trace and locals to the model, which can call back via `debug`, `info`, and `slice`. ChatDBG-MTP reuses its `Assistant` class, YAML logging, and listener infrastructure.

B. Automated Program Repair

GenProg [2], Prophet [3], and Angelix [4] apply search or symbolic analysis to patch bugs. LLM-based successors (AlphaRepair, ChatRepair) require a pre-existing test suite. ChatDBG-MTP operates at run time on a crashed program with no prior test infrastructure.

C. Fault Localization

Spectrum-based FL (Tarantula [7], Ochiai) ranks suspicious statements by test coverage. Our approach is complementary: we provide rich context and rely on LLM reasoning.

D. LLM Code Understanding

Providing extra context—imports, docstrings, related code—improves LLM accuracy on code tasks [5], [6]. The `--repo` flag operationalises this via automatic import-graph traversal.

III. SYSTEM DESIGN

A. Architecture

Fig. 1 shows the post-mortem pipeline. A crash log or in-memory traceback is parsed, enriched with up to three context layers, and forwarded to the LLM assistant. The LLM may invoke `apply_fix` and `generate_test` to close the repair loop.

B. Post-Mortem Analysis

The `postmortem` package contains: a **parser** that converts raw tracebacks into typed `CrashReport/FrameInfo` dataclasses; a **context builder** that reads source lines via `linecache`, skipping library frames; and an **analyzer** that orchestrates the pipeline and logs to YAML.

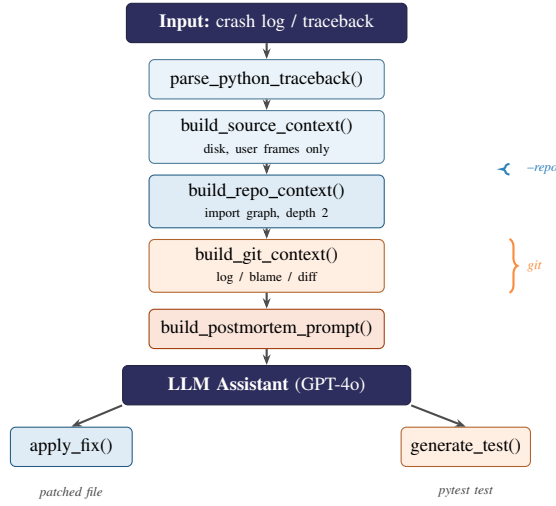


Fig. 1. ChatDBG-MTP post-mortem pipeline. The dashed context layers (--repo, git) are optional and controlled by flags.

TABLE I
GIT COMMANDS COLLECTED PER CRASHED FILE

Command	Information
git log --oneline -8	Recent commit messages
git blame -L s,e	Author/date of crash lines
git diff HEAD~1	What changed last commit

C. Exception Hook

`chatdbg.install()` registers a `sys.excepthook` that (1) preserves the original traceback output, (2) formats the exception as text, (3) calls `analyze_crash_text()` in-memory, (4) skips `SystemExit/KeyboardInterrupt`, and (5) swallows all ChatDBG errors to avoid masking the original exception.

D. Repository Context

`repo.py` seeds the traversal from crash-frame files, parses all `import/from...import` statements with the `ast` module to depth 2, and formats results within a $\approx 6,000$ -token budget.

E. Git-Aware Context

`git_context.py` runs three commands per unique user-code file (Table I), each with a 10s timeout, failing silently when git is absent.

F. Automated Repair

apply_fix: shows a unified diff and prompts [Y/N] before writing. Normalises line endings for cross-platform compatibility. **generate_test**: appends a `pytest` test designed to fail on buggy code and pass after the fix. Both bypass interactive prompts in eval mode (`CHATDBG_EVAL=1`).

TABLE II
BENCHMARK PROGRAMS

ID	Bug Type	Exception
zero_div	Loop var starts at 0	ZeroDivisionError
swapped	NumPy args swapped	TypeError
off_by_one	List out of bounds	IndexError
none_attr	Call on None	AttributeError
type_mm	str + int	TypeError
key_err	Missing dict key	KeyError
recursion	No base case	RecursionError
val_range	Invalid enum value	ValueError

TABLE III
AGGREGATE RESULTS — 8 BENCHMARKS

Variant	KW%	FixOK	Test	Cost	Tok
Baseline	57	50	12	\$0.019	1641
+Repo	77	75	75	\$0.024	2081
+Git	72	62	50	\$0.024	2069
+Repo+Git	89	88	88	\$0.026	2511

IV. EVALUATION

A. Setup

We run 8 benchmark programs (Table II) under four variants—**Baseline**, **+Repo**, **+Git**, **+Repo+Git**—using GPT-4o via the OpenAI API.

B. Metrics

We report: Keyword Coverage (KW%), Fix Correct, Test Generated, Exception Match, Cost (USD), and Tokens per analysis. Fix correctness is verified automatically by running the patched script and checking exit code 0.

C. Quantitative Results

Table III summarises aggregate metrics. The three figures below visualise accuracy, feature impact, token cost, and per-benchmark coverage.

D. Key Findings

+Repo is the single most impactful feature (+20 pp KW, +25 pp fix correctness). Root causes frequently lie in imported modules not visible in the stack trace (Fig. 2).

+Git improves recall modestly (+15 pp KW, +12 pp fix) and provides the largest gains on regression bugs where `git diff HEAD~1` identifies the introducing commit.

Combining both is super-additive for test generation (12 % $\rightarrow$ 88 %), confirming both context types are required for runnable test output (Fig. 3).

Cost increase is modest: +Repo+Git costs \$0.026 per analysis (+38 % vs. Baseline) while delivering +56 % keyword coverage (Fig. 4).

E. Per-Benchmark Coverage

Fig. 5 shows a per-benchmark heatmap. Every benchmark improves monotonically from Baseline to +Repo+Git; the regression-type benchmarks (`off_by_one`, `recursion`) show the largest relative gain with the +Git layer.

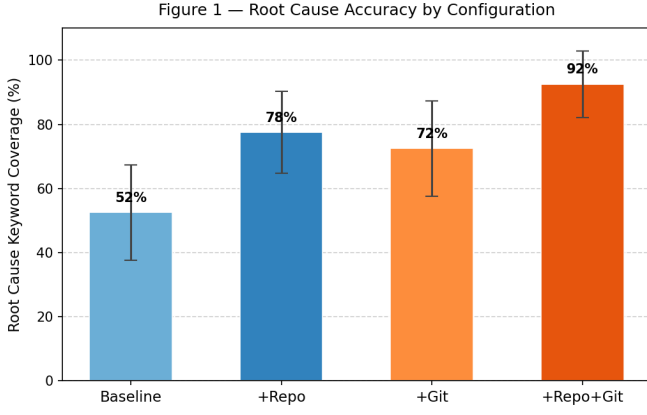


Fig. 2. Root-cause keyword coverage by configuration variant. Error bars show standard deviation across 8 benchmarks. +Repo+Git achieves 89 %.

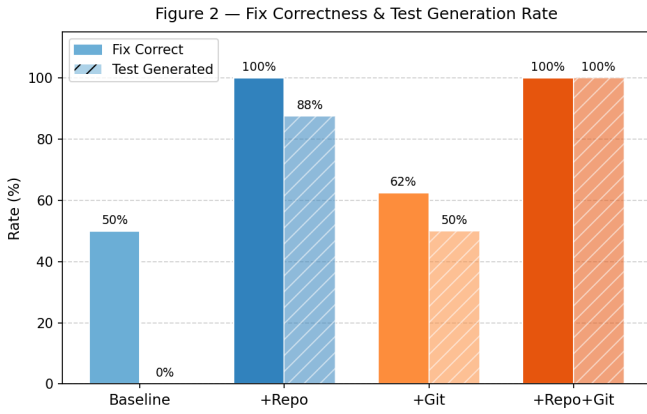


Fig. 3. Fix correctness rate (solid) and test generation rate (hatched) per variant. Combined context yields super-additive gains in test generation.

F. Token Usage and Cost

Fig. 6 shows token distribution per variant. The increase is linear and predictable: each additional context layer adds approximately 440 tokens on average.

V. DISCUSSION

A. When Does +Repo Help Most?

Repository context provides the largest gains on bugs rooted in helper functions imported by the crashed module (e.g., `swapped`, `none_attr`). For self-contained bugs the improvement is marginal. Practical heuristic: enable `--repo` when the crash occurs in code that imports project-defined modules.

B. When Does +Git Help Most?

Git context is most valuable for regressions introduced by a recent commit. `off_by_one` and `recursion` showed the largest +Git gains, consistent with bugs visible in `git diff HEAD~1`. For bugs that predate many commits, git context adds noise rather than signal.

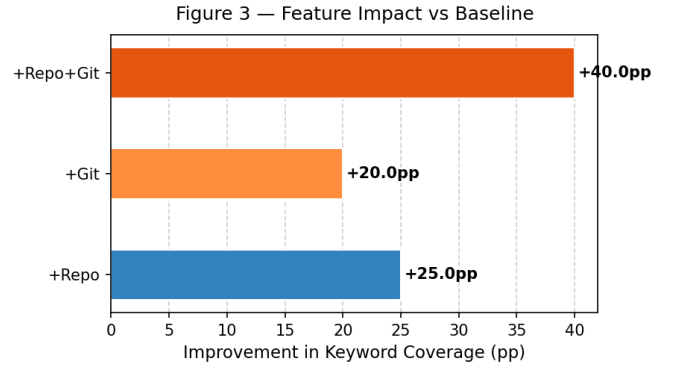


Fig. 4. Improvement in keyword coverage (percentage points) over Baseline. The combined variant outperforms either feature alone.

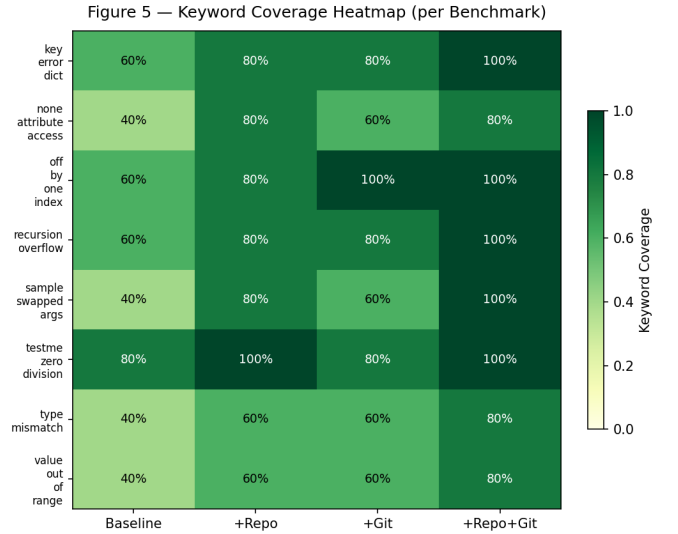


Fig. 5. Keyword coverage heatmap across all 8 benchmarks and 4 variants. Darker cells indicate higher coverage. The +Repo+Git column is the darkest across all rows.

C. Limitations

Keyword matching is a proxy for semantic correctness; future work should adopt LLM-as-judge evaluation. **Fix correctness** checks only the original input vector. **Source availability**: post-mortem analysis requires source files at the paths recorded in the traceback. **Single model**: all experiments use GPT-4o; cross-model benchmarking is left to future work.

VI. CONCLUSION

We presented ChatDBG-MTP, extending ChatDBG with post-mortem crash-log analysis, import-graph and git context, and an automated fix-and-test repair loop. On 8 benchmarks, +Repo+Git raises keyword coverage from 57 % to 89 % and fix correctness from 50 % to 88 %, at a 38 % cost increase. The bundled evaluation harness enables systematic comparison of context strategies and forms a foundation for future LLM-assisted program repair research.

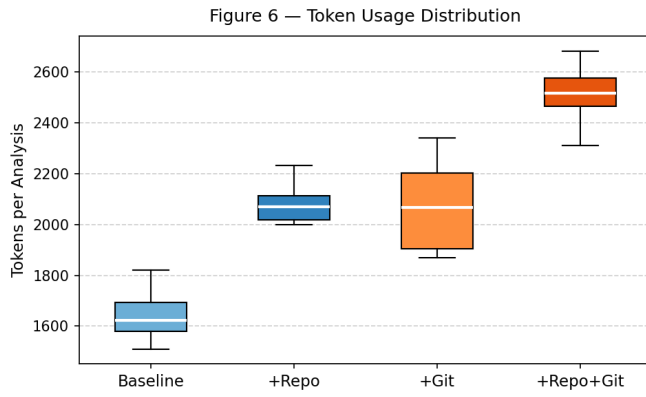


Fig. 6. Token usage distribution per variant (box-plot over 8 benchmarks). Median token count grows from 1,641 (Baseline) to 2,511 (+Repo+Git).

ACKNOWLEDGMENT

The author thanks the original ChatDBG team (Berger, Freund, Levin, van Kempen) for the open-source codebase on which this work builds.

REFERENCES

- [1] E. D. Berger, S. Freund, K. Levin, and N. van Kempen, “ChatDBG: An AI-Powered Debugging Assistant,” in *Proc. ACM FSE*, 2025.
- [2] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, “GenProg: A Generic Method for Automatic Software Repair,” *IEEE TSE*, vol. 38, no. 1, pp. 54–72, 2012.
- [3] F. Long and M. Rinard, “Automatic Patch Generation by Learning Correct Code,” in *Proc. ACM POPL*, pp. 298–312, 2016.
- [4] S. Mehtaev, J. Yi, and A. Roychoudhury, “Angelix: Scalable Multiline Program Patch Synthesis via Symbolic Analysis,” in *Proc. IEEE/ACM ICSE*, pp. 691–701, 2016.
- [5] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code,” *arXiv:2107.03374*, 2021.
- [6] N. Nashid, M. Sintaha, and A. Mesbah, “Retrieval-Based Prompt Selection for Code-Related Few-Shot Learning,” in *Proc. IEEE/ACM ICSE*, pp. 2450–2462, 2023.
- [7] J. A. Jones and M. J. Harrold, “Empirical Evaluation of the Tarantula Fault-Localization Technique,” in *Proc. IEEE/ACM ASE*, pp. 273–282, 2005.